

Influencer or Observer: Predicting Social Roles

**SENG Hok & TAGNE WAFFO Joel &
DEHAYEM KENFOUO Sylvain**

15 décembre 2025

CSC_51054_EP
École polytechnique

Data Preprocessing and Feature Selection

Preprocessing

- We removed feature with zero variance
- There are 73 % of features had more than 50% missing values. We removed features whose missing values are more than 90%
- There are around 38K users indicated by the description, but we identified 30K using `user.created_at`
- We fill some missing values, we group our data according to the `user.created_at` then used median for numerical data and the most frequent element for others.

FEATURE ENGINEERING

User level	Tweet level	User level aggregated
account_age	tweet_full_text	mean_account_age_days
total_activity	user_description_length	mean_user_listed,
statuses_per_day,		mean_total_activity
followers_per_day		mean_text_length,
tweet_hour,		mean_user_description_length
tweet_dayofweek		mean_tweet_hour,
		mean_tweetdayofweek

Table 1: User-level, tweet-level features, and their user-level aggregations.

- `account_age = tweet_datetime - user_created_datetime`
- `total_activity = user.statuses_count +
user.favourites_count + user.listed_count`

About feature selection

- We did Principal component analysis to visualize the distribution of data.

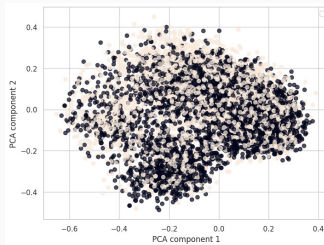


Figure 1: Plot obtained using 2 components PCA

About feature selection

- We did correlation analysis for the numerical features to find out which feature are highly correlated.

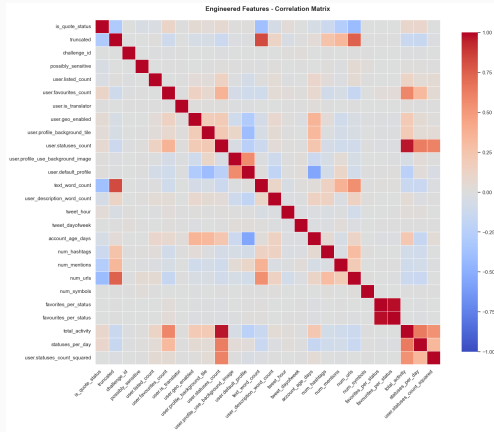


Figure 2: Correlation map after data preprocessing

Model & Training

Model pipeline



Text Encoder

- TF-IDF encoder
- Train a Bert model for classification (Bert, Roberta, Bertweet).
 - Gather informations about the tweet in the text.
 - Try LoRA fine-tuning

$$W' = W + \Delta W, \quad \Delta W = BA,$$

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r},$$

- Full fine-tuning

Text Encoder

- TF-IDF encoder
- Train a Bert model for classification (Bert, Roberta, Bertweet).
 - Gather informations about the tweet in the text.
 - Try LoRA fine-tuning

$$W' = W + \Delta W, \quad \Delta W = BA,$$

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r},$$

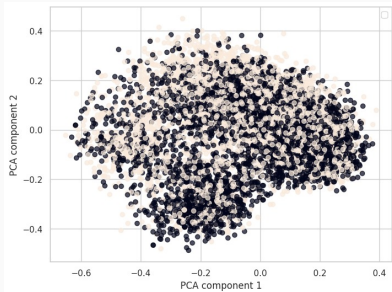
- Full fine-tuning
- Autoregressive Training of pythia :
 - Prompt :

What type of Twitter account posted this tweet?

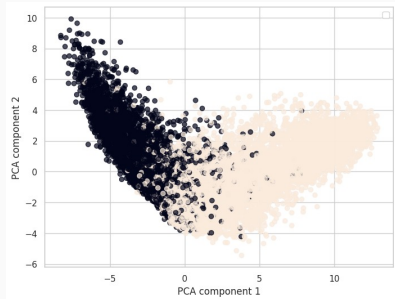
'combined_text' OPTIONS: 0: Observer 1: Influencer

ANSWER: 'label'

Text Encoder



TF-IDF PCA



Trained Bert PCA

- Multilayer Perceptron (MLP)
- Random Forest (RF)
- LightGBM
- XGBoost
- Hyperparameter optimization with Optuna
 - Learning rate
 - Max depth of trees
 - regularization parameters
 - numbers leaves

Model Tuning and results

Model comparison and experimental setup

- Text data encoded using pretrained language models :
 - **Pythia-160M** (GPT-like autoregressive LLM)
 - **XLM-RoBERTa-base** (BERT-like bidirectional encoder)
- Text embeddings used as fixed features (feature-based transfer learning)
- Final predictions obtained using tree-based boosting models :
 - **LightGBM** (Gradient Boosting Decision Trees)
 - **XGBoost** (Extreme Gradient Boosting)
- Models evaluated using Stratified Group K-Fold (user-level split)

- **step 1 : Cross-validated probability averaging**
 - Single model : **XGBoost, LightGBM**
 - Same model trained on multiple folds
 - Predicted probabilities averaged across folds

$$\hat{p}(x) = \frac{1}{K} \sum_{k=1}^K \hat{p}_k^{\text{Mod}}(x), \quad \hat{y} = 1_{[\hat{p}(x) \geq 0.5]} \quad (1)$$

- **Step 2 : User-level aggregation**
 - Predictions aggregated by user identifier (`user.created_at`)
 - Ensures a consistent prediction for all tweets from the same user
 - Reduces prediction noise at the tweet level

Ensemble strategies

- step 3 :

- Different models rely on different explanatory signals
- This reduces correlation between model predictions
- Final predictions are obtained using a majority vote between decorrelated models

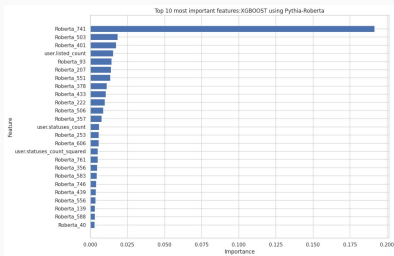


Figure 3: XGBoost feature importance (Pythia + RoBERTa embeddings dominant)

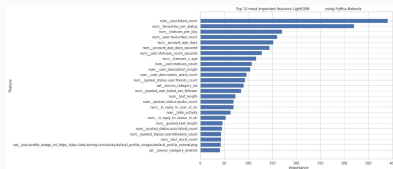


Figure 4: LightGBM feature importance (behavioral features dominant)

$$\hat{y} = 1[\sum_{m=1}^3 \hat{y}_m > 1.5] \quad (2)$$

Model	Accuracy
XGB + Roberta 2k+ Pythia 6k	0.845
XGB + Roberta 0k+ Pythia 6k	0.845
XGB + Bert LORA Finetuning	0.843
LGBM + Roberta 2k+ Pythia 6k	0.846
LGBM + Roberta 3K+ Pythia 6k (with user features)	0.856
LGBM + Roberta 0k+ Pythia 6k	0.846
LGBM + Roberta 0k+ Pythia 6k (with user features)	0.852
LGBM + Roberta 2k	0.843
LGBM + Roberta 2k (with user features)	0.853
LGBM + Pythia 0k	0.840
LGBM + TFIDF (with user features)	0.850
LGBM + Bert LORA finetuning	0.843
LGBM + Bert Tweet 0k+ Pythia 6k (with user features)	0.853
Final ensemble (LGBM + XGBoost+ (Roberta 3K + Pythia 6K+ Bert Tweet 0k))	0.859

Figure 5: Comparison of model performances with different embeddings and classifiers.